Computing Delaunay Triangulations

1. Software for Delaunay Triangulations
   - the spatial module of scipy
   - using CGAL in Python
   - Triangle: mesh generator and triangulator
2. The Delaunay Graph
   - the dual of a Voronoi diagram
   - angle optimal triangulations
3. Computing the Delaunay Triangulation
   - a randomized incremental algorithm
   - formulation of the algorithms

MCS 481 Lecture 28
Computational Geometry
Jan Verschelde, 28 October 2026

Computing Delaunay Triangulations

1. Software for Delaunay Triangulations
   - the spatial module of scipy
   - using CGAL in Python
   - Triangle: mesh generator and triangulator
2. The Delaunay Graph
   - the dual of a Voronoi diagram
   - angle optimal triangulations
3. Computing the Delaunay Triangulation
   - a randomized incremental algorithm
   - formulation of the algorithms

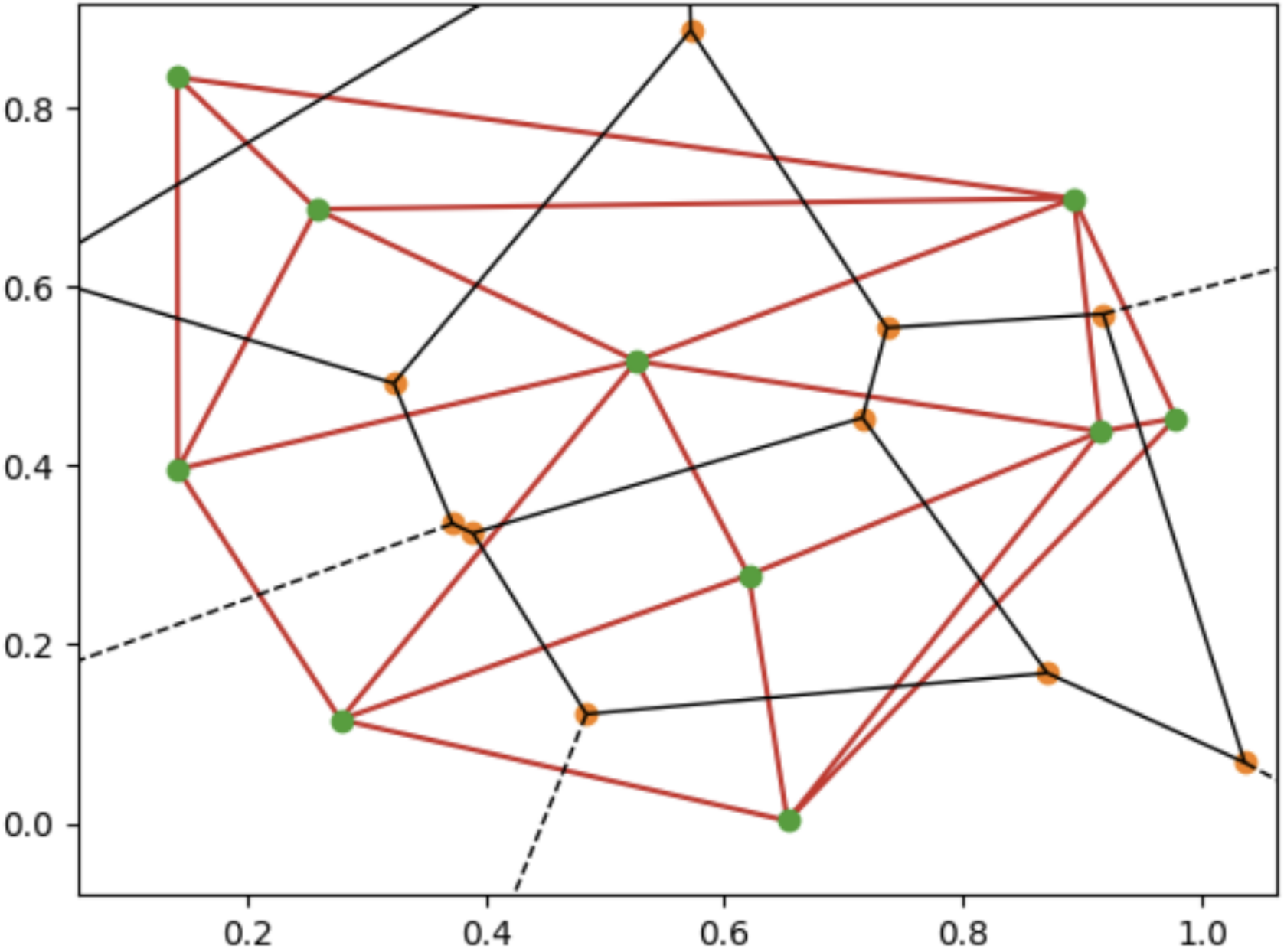
Voronoi Diagram and Delaunay Triangulation

To illustrate duality, consider 10 random points.

```
import numpy as np
from scipy.spatial import Voronoi, voronoi_plot_2d
from scipy.spatial import Delaunay, delaunay_plot_2d
from matplotlib import pyplot as plt

points = np.random.rand(10, 2)
vor = Voronoi(points)
tri = Delaunay(points)
fig, axs = plt.subplots()
voronoi_plot_2d(vor, ax=axs)
delaunay_plot_2d(tri, ax=axs)
plt.show()
```

Voronoi Diagram and Delaunay Triangulation



Computing Delaunay Triangulations

1. Software for Delaunay Triangulations
   - the spatial module of scipy
   - using CGAL in Python
   - Triangle: mesh generator and triangulator
2. The Delaunay Graph
   - the dual of a Voronoi diagram
   - angle optimal triangulations
3. Computing the Delaunay Triangulation
   - a randomized incremental algorithm
   - formulation of the algorithms

using CGAL in Python

The CGAL code to construct Delaunay triangulation are wrapped.

```
from CGAL.CGAL_Triangulation_2 \
import Delaunay_Triangulation_2
```

See the Jupyter notebook.

Observe that the `T.insert(point)` step,
the algorithm is *incremental*, it adds one point after another.

Exercise 1: Illustrate the duality between the Voronoi diagram and the Delaunay triangulation using an example computed with CGAL.

Computing Delaunay Triangulations

1. Software for Delaunay Triangulations
   - the spatial module of scipy
   - using CGAL in Python
   - Triangle: mesh generator and triangulator
2. The Delaunay Graph
   - the dual of a Voronoi diagram
   - angle optimal triangulations
3. Computing the Delaunay Triangulation
   - a randomized incremental algorithm
   - formulation of the algorithms

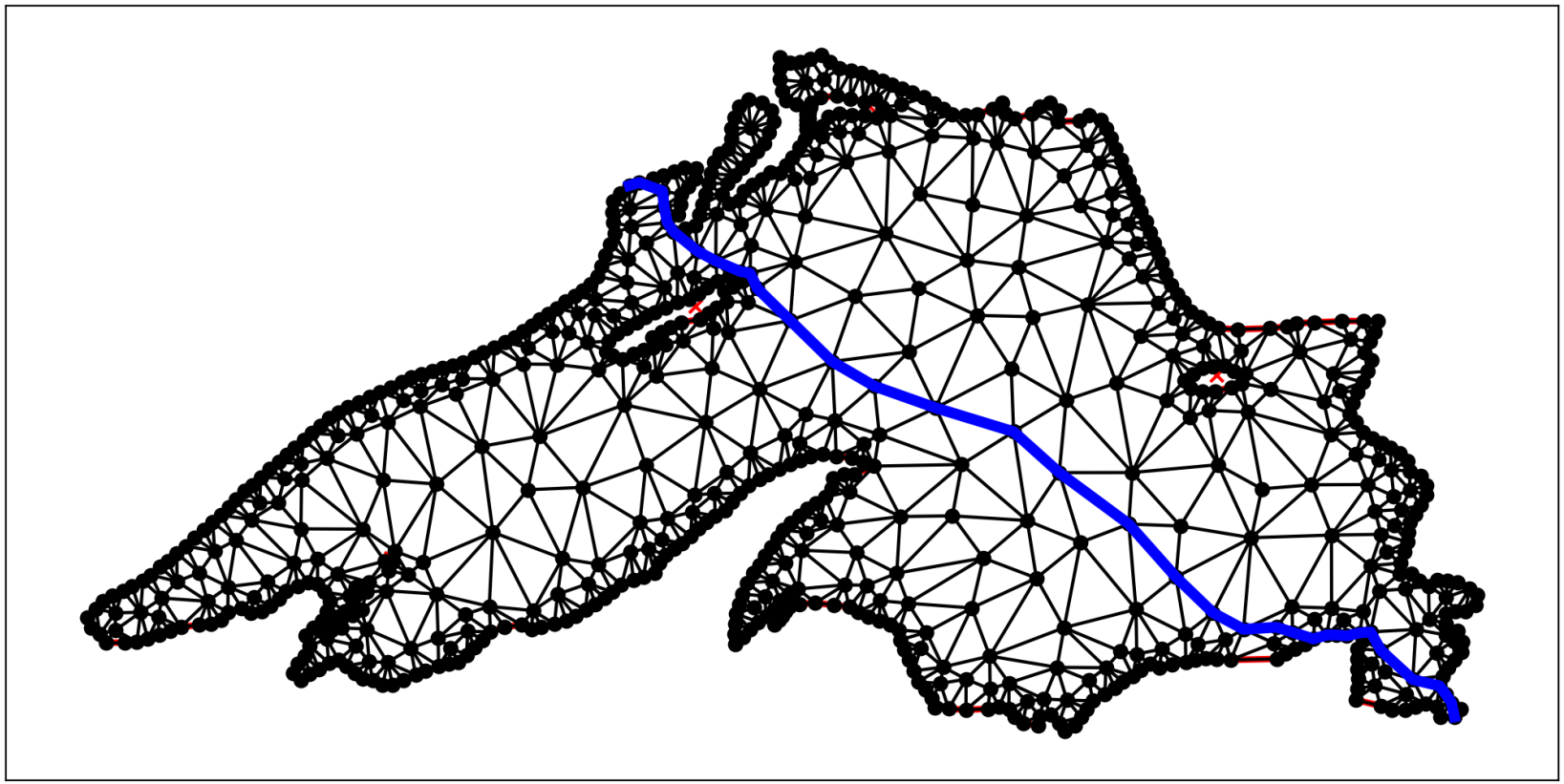
prize winning software

Jonathan Richard Shewchuk, ***Triangle: Engineering a 2D Quality Mesh Generator and Delaunay Triangulator***,
in *Applied Computational Geometry: Towards Geometric Engineering*, edited by Ming C. Lin and Dinesh Manocha, volume 1148 of Lecture Notes in Computer Science, pages 203-222, Springer-Verlag, Berlin, May 1996.

Received the 2003 James H. Wilkinson Prize for Numerical Software.

Usable in Python through wrappers written by Dzhelil Rufat, available via `pip install triangle`.

a shortest path in the lake Superior polygon



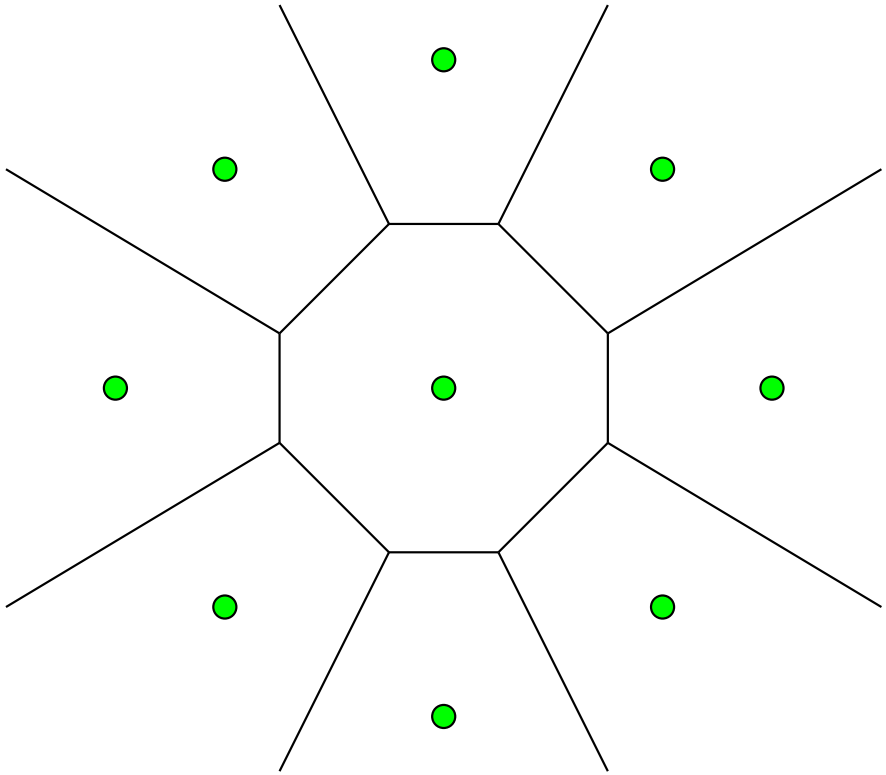
From *Mastering SciPy* by Francisco J. Blanco-Silva, Packt Publishing 2015.

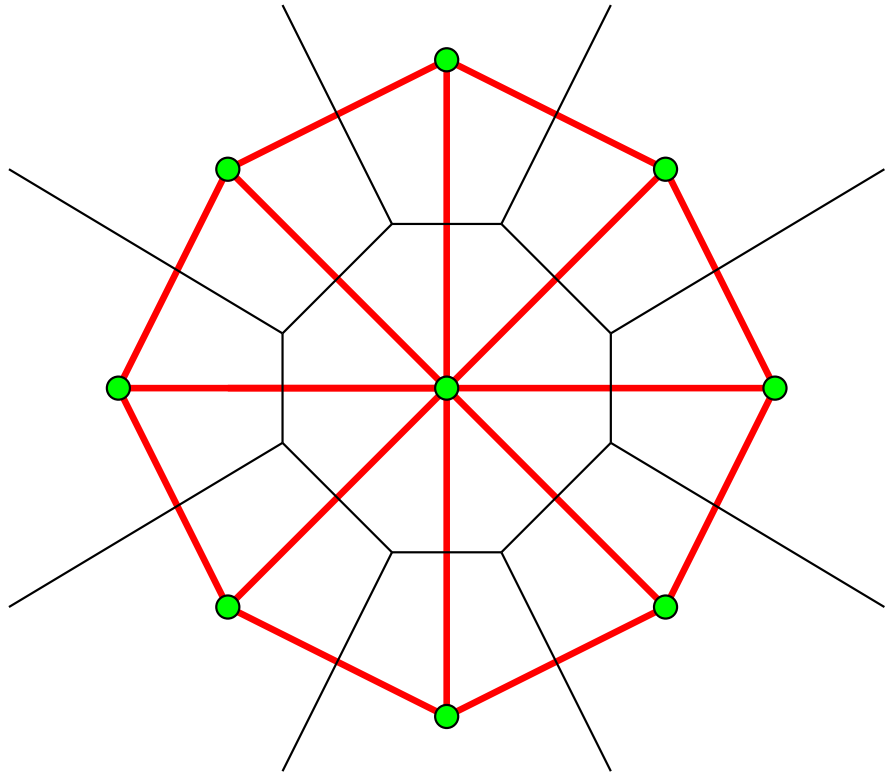
Computing Delaunay Triangulations

1. Software for Delaunay Triangulations
   - the spatial module of scipy
   - using CGAL in Python
   - Triangle: mesh generator and triangulator
2. The Delaunay Graph
   - the dual of a Voronoi diagram
   - angle optimal triangulations
3. Computing the Delaunay Triangulation
   - a randomized incremental algorithm
   - formulation of the algorithms

the dual of a Voronoi diagram

At the left is a Voronoi diagram.





At the right is the dual of the Voronoi diagram.

the dual graph of a Voronoi diagram

Definition (the dual graph of a Voronoi diagram)

Let $\mathrm{Vor}(P)$ be the Voronoi diagram of the set of points P.
The *dual graph* $\mathcal{DG}(P)$ *of* $\mathrm{Vor}(P)$ is constructed as follows:

- the vertices are the sites of $\mathrm{Vor}(P)$, and
- two vertices are connected
  if and only if the Voronoi cells of their sites share an edge.

To every vertex of $\mathrm{Vor}(P)$ corresponds a face of $\mathcal{DG}(P)$.
To every face of $\mathcal{DG}(P)$ corresponds a vertex of $\mathrm{Vor}(P)$.

Theorem (planarity of the dual graph)

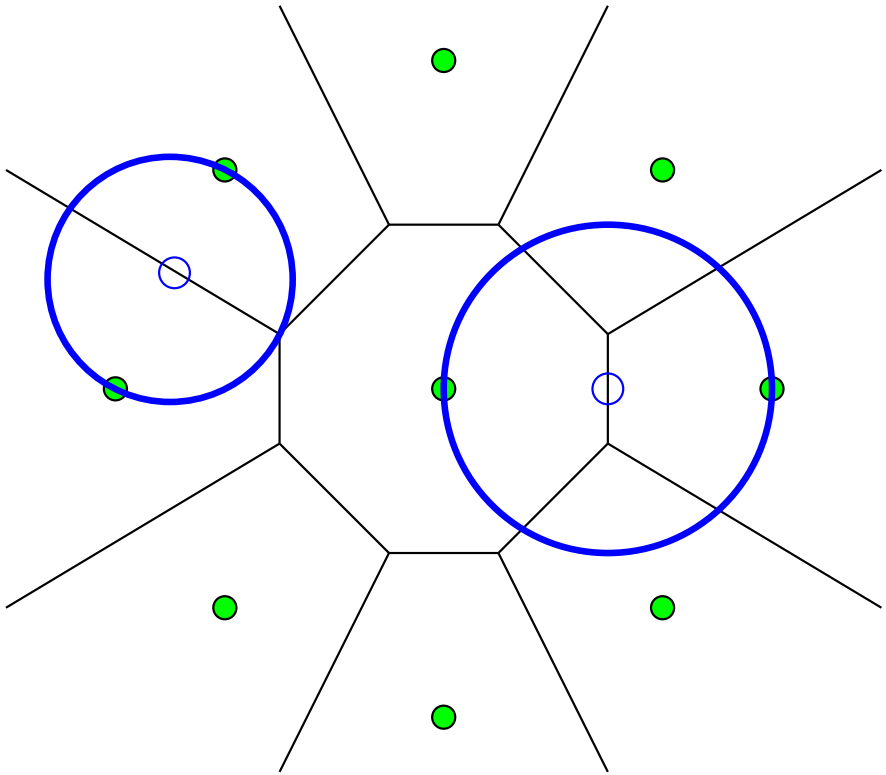
The graph $\mathcal{DG}(P)$ *is planar.*

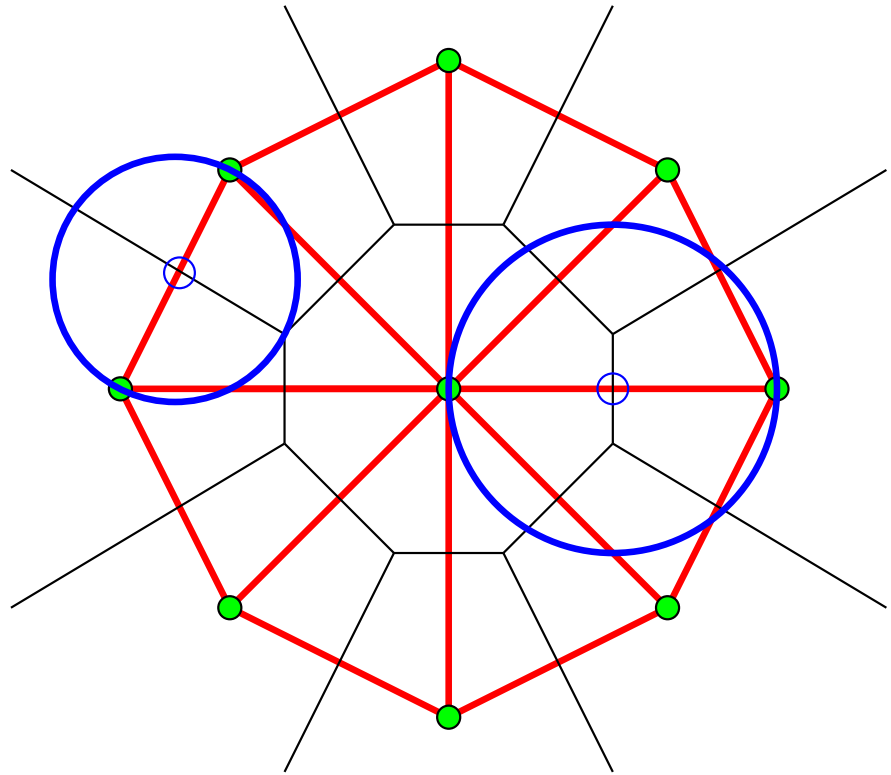
In a planar graph, there are no crossing edges.

a property of edges of the dual graph

For every edge (p, q) of $\mathcal{DG}(P)$, there is a circle C

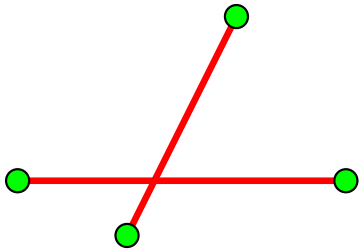
- with p and q on the boundary of C, and
- no site of P lies in C.

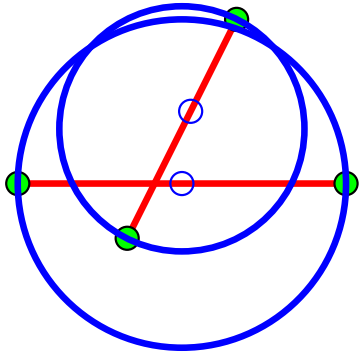




there are no crossing edges

By contradiction, assume two edges cross:





But then the circle of one edge must contain a vertex in its interior.

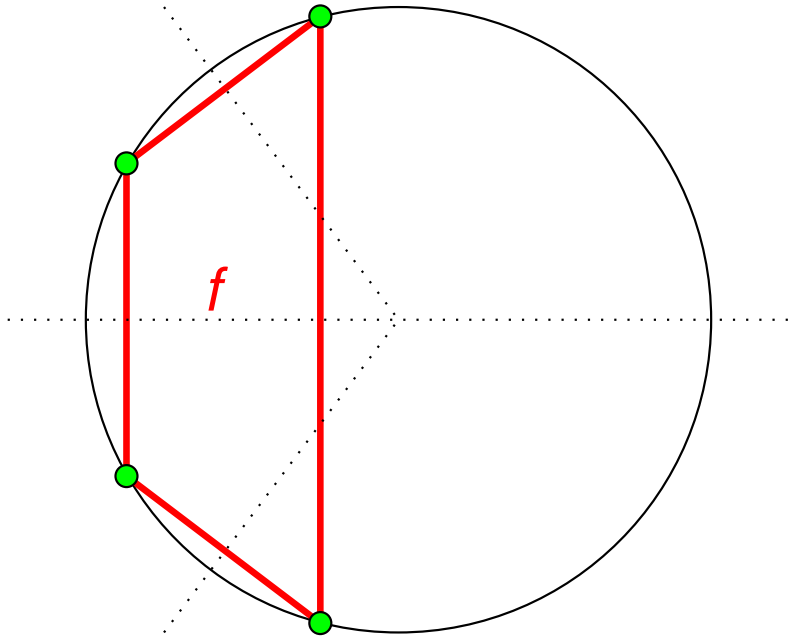
This gives us the contradiction, so our assumption is wrong.

Q.E.D.

points not in general position

We say that P is *in general position*
if the degree of every vertex in $\mathrm{Vor}(P)$ is three.

If P is *not* in general position,
then some face f in $\mathcal{DG}(P)$ is not a triangle.



Since f is convex, adding edges to obtain a triangulation is easy.

uniqueness

We derived a criterion for when we can speak of
the Delaunay triangulation.

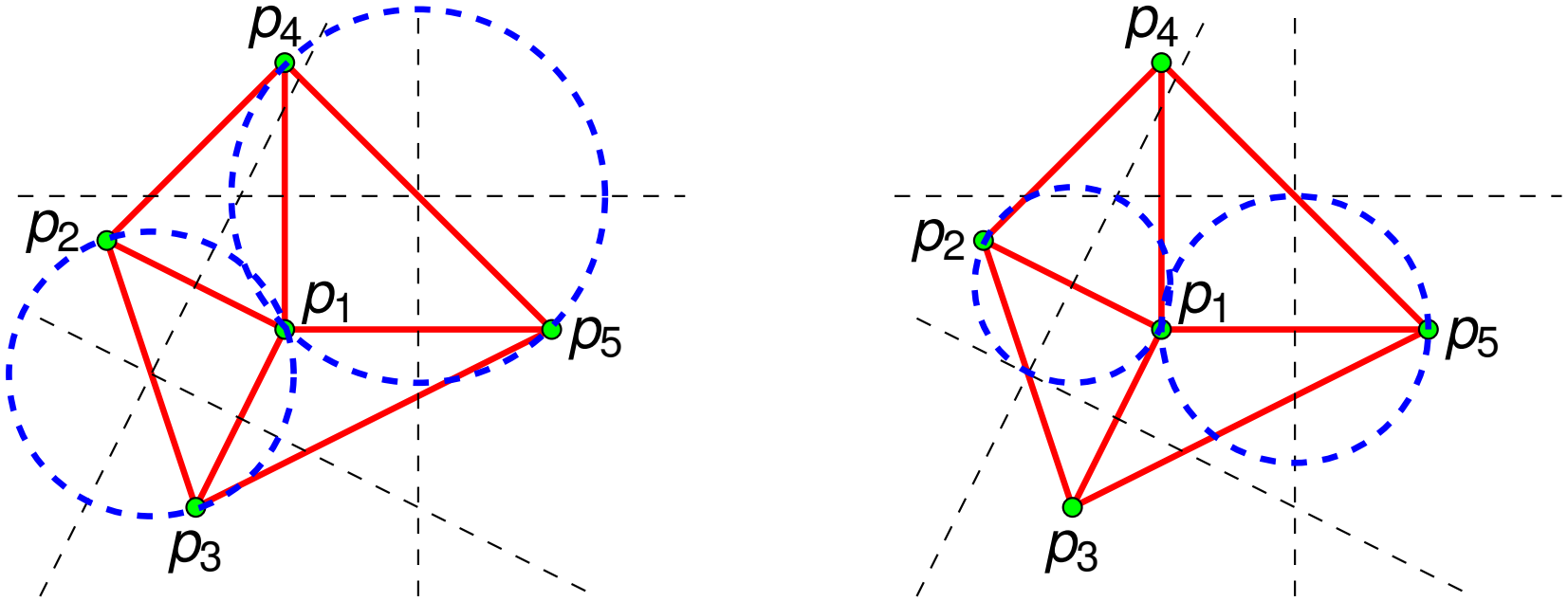
Proposition (uniqueness)

The Delaunay triangulation of a point set P is unique if and only if P is in general position.

Computing Delaunay Triangulations

1. Software for Delaunay Triangulations
   - the spatial module of scipy
   - using CGAL in Python
   - Triangle: mesh generator and triangulator
2. The Delaunay Graph
   - the dual of a Voronoi diagram
   - angle optimal triangulations
3. Computing the Delaunay Triangulation
   - a randomized incremental algorithm
   - formulation of the algorithms

characterization of vertices and edges



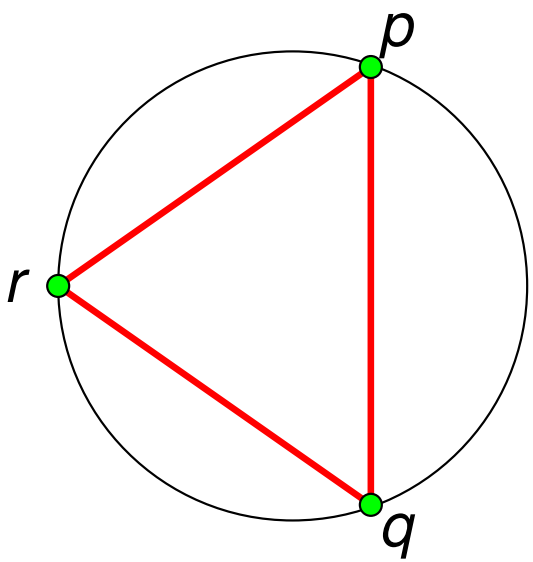
Theorem (characterization of vertices and edges)

Let P be a set of points in the plane.

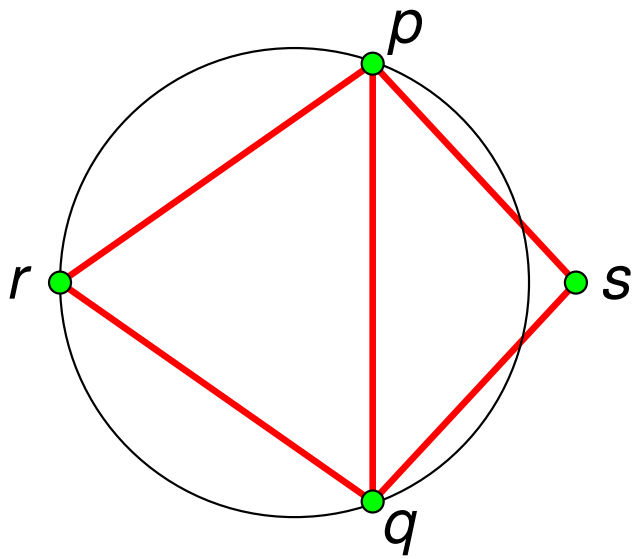
- *$p_i, p_j, p_k \in P$ are vertices of $\mathcal{DG}(P)$ $\Leftrightarrow$ the circle through p_i, p_j, p_k contains no point of P in its interior.*
- *$p_i, p_j \in P$ form an edge of $\mathcal{DG}(P) \Leftrightarrow$ a circle C contains p_i and p_j on its boundary and no other point of P lies inside C.*

circumcircle characterization

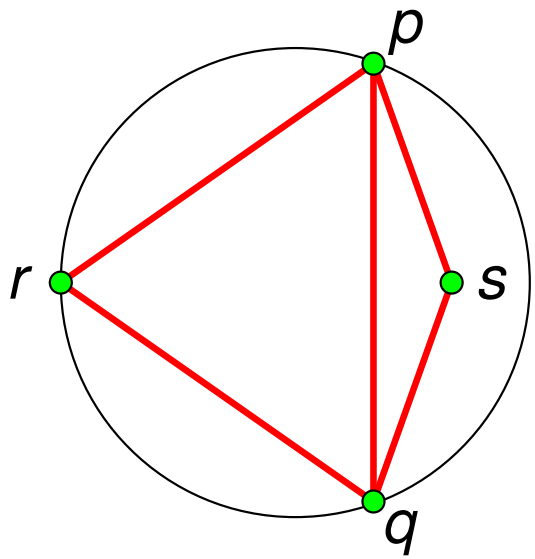
The three vertices of a triangle define a circle uniquely.



circumcircle



legal



illegal

Theorem (circumcircle characterization)

A triangulation T of P is Delaunay $\Leftrightarrow$ the circumcircle of any triangle of T does not contain a point of P in its interior.

characterization of Delaunay triangulations

By flipping illegal edges we turn any triangulation into a legal one.

Theorem (legal triangulations)

Let P be a set of points in the plane.
A triangulation T is legal $\Leftrightarrow$ T is a Delaunay triangulation.

The theory is summarized in the following theorem:

Theorem (angle optimal triangulations are Delaunay)

Any angle optimal triangulation is Delaunay and
any Delaunay triangulation is angle optimal.

The above theorem justifies the definition of a Delaunay triangulation.

Computing Delaunay Triangulations

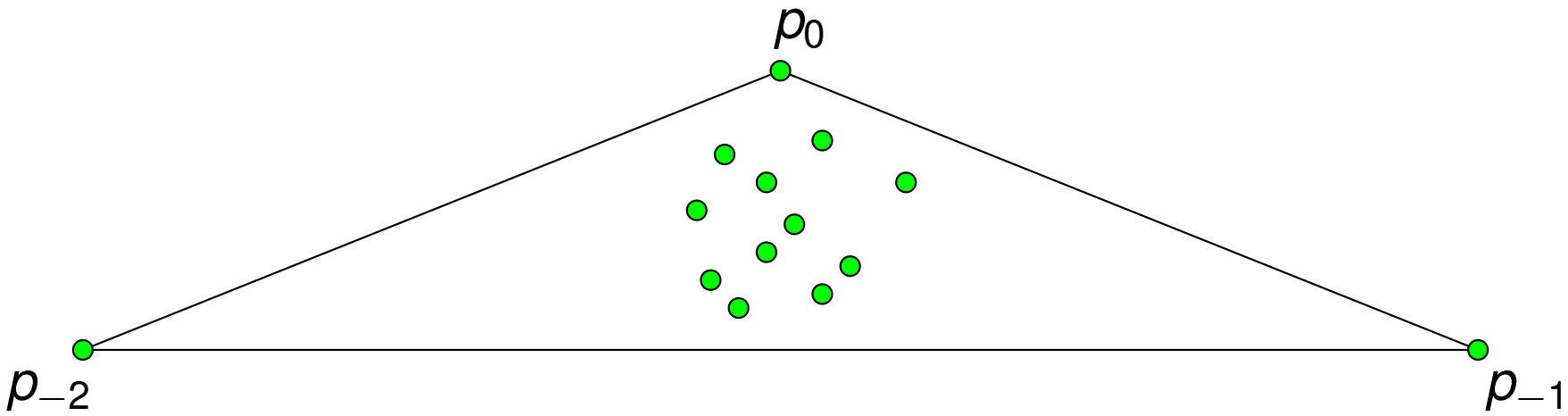
1. Software for Delaunay Triangulations
   - the spatial module of scipy
   - using CGAL in Python
   - Triangle: mesh generator and triangulator
2. The Delaunay Graph
   - the dual of a Voronoi diagram
   - angle optimal triangulations
3. Computing the Delaunay Triangulation
   - a randomized incremental algorithm
   - formulation of the algorithms

the initial triangle

Instead of applying edge flips to an existing triangulation, we will compute the Delaunay triangulation directly.

The initial triangle is defined as follows:

1. Let p_0 be the highest point of P.
2. Choose p_{-1} and p_{-2} so they do not lie on any circle defined by three points of P.

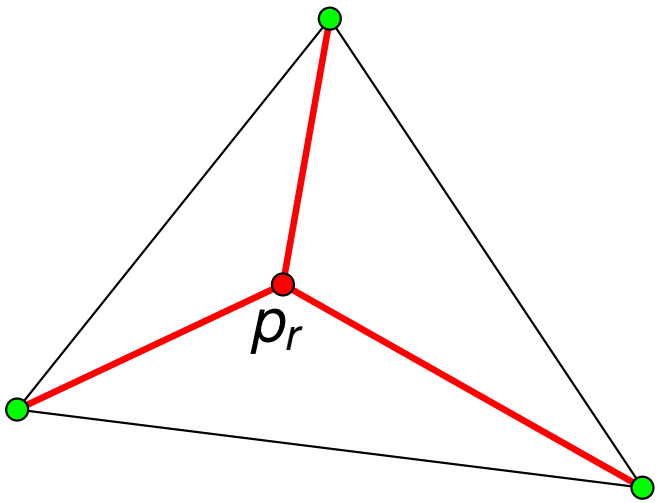


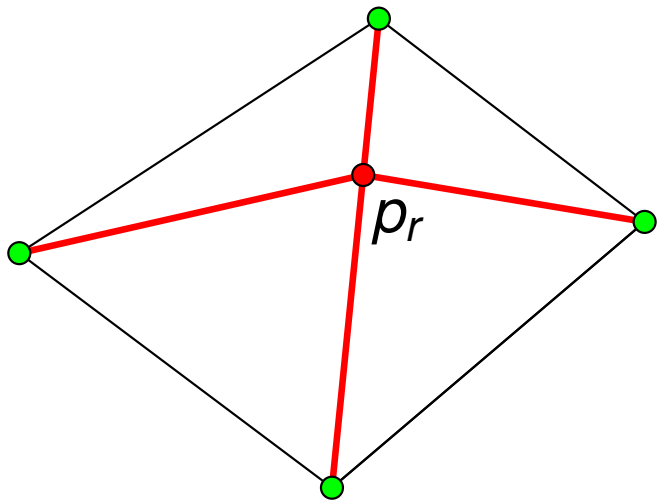
adding a point

Adding a point p_r:

1. locate the triangle Δ that contains p_r,
2. add edges from p_r to the vertices of Δ.

Special care for when p_r lies on an edge shared between two triangles.





In every new triangle, illegal edges are replaced via edge flips.

Computing Delaunay Triangulations

1. Software for Delaunay Triangulations
   - the spatial module of scipy
   - using CGAL in Python
   - Triangle: mesh generator and triangulator
2. The Delaunay Graph
   - the dual of a Voronoi diagram
   - angle optimal triangulations
3. Computing the Delaunay Triangulation
   - a randomized incremental algorithm
   - formulation of the algorithms

making edges legal

Algorithm LEGALIZEEDGE($T, p_r, (p_i, p_j)$)

Input: T is a Delaunay triangulation,
the point p_r and edge (p_i, p_j) define a *new* triangle.
Output: T with the illegal edges flipped.

1. if the edge (p_i, p_j) is illegal then
2. let $\Delta p_i p_j p_k$ be the triangle adjacent to $\Delta p_i p_j p_r$
3. replace in T the edge (p_i, p_j) by the edge (p_r, p_k)
4. LEGALIZEEDGE($T, p_r, (p_i, p_k)$)
5. LEGALIZEEDGE($T, p_r, (p_j, p_k)$)

Exercise 2: Make a drawing to justify the steps in the algorithm.
Why are steps 4 and 5 needed?
Why does LEGALIZEEDGE terminate?

a randomized incremental algorithm

Algorithm DELAUNAYTRIANGULATION(P)

Input: P, a set of $n+1$ points, $P = \{ p_0, p_1, \ldots, p_n \}$.
Output: T, a Delaunay triangulation of P.

1. let p_0 be the rightmost highest point in P
2. let p_{-1} and p_{-2} be such that $P \subseteq \Delta p_{-2} p_{-1} p_0$
3. $T = \{ \Delta p_{-2} p_{-1} p_0 \}$
4. compute a random permutation of $P \setminus \{ p_0 \}$
5. for r from 1 to n do
6. locate $\Delta p_i p_j p_k \in T$: $p_r \in \Delta p_i p_j p_k$
7. UPDATE($T, p_r, \Delta p_i p_j p_k$)

updating the triangulation

Algorithm UPDATE($T, p_r, \Delta p_i p_j p_k$)

Input: T, a Delaunay triangulation, $p_r \in \Delta p_i p_j p_k \in T$.
Output: T, a Delaunay triangulation, with p_r added.

1. if p_r belongs to the interior of $\Delta p_i p_j p_k$ then
2. add edges (p_i, p_r), (p_j, p_r), (p_k, p_r) to T
3. LEGALIZEEDGE($T, p_r, (p_i, p_j)$)
4. LEGALIZEEDGE($T, p_r, (p_j, p_k)$)
5. LEGALIZEEDGE($T, p_r, (p_k, p_i)$)

Exercise 3: Make a drawing to justify the second step in the algorithm. How can we be sure steps 4 and 5 do not undo the work of step 3?

updating the triangulation – continued

1. if p_r belongs to the interior of $\Delta p_i p_j p_k$ then
2. $\quad$ ... *(steps on the previous slide)*
6. if p_r lies on the edge (p_i, p_j) of $\Delta p_i p_j p_k$ then
7. $\quad$ add edges (p_i, p_r), (p_j, p_r), (p_k, p_r), (p_k, p_ℓ) to T
8. $\quad$ LEGALIZEEDGE$(T, p_r, (p_i, p_\ell))$
9. $\quad$ LEGALIZEEDGE$(T, p_r, (p_\ell, p_j))$
10. $\quad$ LEGALIZEEDGE$(T, p_r, (p_k, p_j))$
11. $\quad$ LEGALIZEEDGE$(T, p_r, (p_i, p_k))$

Make a drawing to justify the steps in the algorithm.

summary and exercises

The Delaunay triangulation is unique for points in general position.
We defined the Delaunay graph: the dual graph of a Voronoi diagram, and outlined an incremental algorithm for the Delaunay triangulation.

We covered section 9.2 and half of 9.3 in the textbook.

Consider the following activities, listed below.

1. Write the solutions to exercises 1, 2, and 3.
2. Consult the CGAL documentation and example code on Delaunay triangulations.
3. Consider the exercises 5, 6, 8 in the textbook.